

MySQL

Para que el servidor nos muestre las bases de datos existentes, se lo solicitamos enviando la instrucción:

```
SHOW DATABASES;
```

Para ver las tablas existentes en una base de datos

```
SHOW TABLES;
```

Para ver la estructura de una tabla usamos el comando "describe" junto al nombre de la tabla:

```
DESCRIBE USUARIOS;
```



Tipos de datos blob y text.

Los tipos "blob" o "text" son bloques de datos.

-La diferencia entre "blob" y "text" es que "text" diferencia mayúsculas y minúsculas

-Tienen una longitud de 65535 caracteres.

- No permiten valores "default".

Existen subtipos:

- tinyblob o tinytext: longitud máxima de 255

- mediumblob o mediumtext: longitud de 16777215

- longblob o longtext: longitud para 4294967295



Tipos de datos blob y text.

Los tipos "blob" o "text" son bloques de datos.

```
CREATE TABLE electrodomesticos (  
    producto varchar(30), marca varchar(20),  
    descripcion text );
```

```
INSERT INTO electrodomesticos values (  
    'heladera con freezer', 'Patrick', 'Capacidad  
    365 litros; Nuevo sistema de frío skin;  
    Anaqueles regulables en altura; Hieleras;  
    Hueveras; Descongelamiento automático  
    cíclico');
```

```
SELECT * from electrodomesticos where  
descripcion like '%litros%'
```



Tipos de datos enum.

El tipo de dato "enum" representa una enumeración.

Puede tener un máximo de 65535 valores distintos.

Es una cadena cuyo valor se elige de una lista enumerada de valores permitidos que se especifica al definir el campo.

Puede ser una cadena vacía, incluso "null".

```
CREATE TABLE postulantes( numero int  
unsigned auto_increment, documento  
char(8), nombre varchar(30), estudios  
enum('ninguno','primario','secundario',  
'terciario','universitario'), primary  
key(numero) );
```



Práctica en Clase

Creamos una base de datos llamada ejercicios

```
CREATE DATABASE `ejercicios` ;
```

Creamos una tabla llamada usuarios con dos campos (nombre y clave).

```
CREATE TABLE usuarios ( nombre  
varchar(20), clave varchar(10), primary  
key(nombre));
```



Agregando Clave Primaria

Si tenemos la tabla "usuarios", el nombre de cada usuario puede establecerse como clave primaria, es un valor que no se repetirá; puede haber usuarios con igual clave, pero su nombre de usuario será siempre distinto.

Para definir un campo como clave primaria agregamos "primary key" luego de la definición de todos los campos y entre paréntesis colocamos el nombre del campo que queremos como clave.

```
CREATE TABLE usuarios ( nombre  
varchar(20), clave varchar(10), primary  
key(nombre));
```

- Los valores no se repiten ni pueden ser nulos.
- Una tabla sólo puede tener una clave primaria.



Claves primarias compuestas

patente char(6) not null,
tipo char (4),
horallegada time not null,
horasalida time,

Necesitamos definir una clave primaria para una tabla con los datos descritos arriba. No podemos usar la patente porque un mismo auto puede ingresar más de una vez en el día a la playa; tampoco podemos usar la hora de entrada porque varios autos pueden ingresar a una misma hora. Tampoco sirven los otros campos.

Definimos una clave compuesta cuando ningún campo por si solo cumple con la condición para ser clave.

Un auto puede ingresar varias veces en un día a la playa, pero siempre será a distinta hora.

Usamos **2 campos como clave**, la patente junto con la hora de llegada, así identificamos unívocamente cada registro.



Claves primarias compuestas

Para establecer más de un campo como clave primaria usamos la siguiente sintaxis:

```
CREATE TABLE vehiculos(  
  patente char(6) not null,  
  tipo char(4),  
  horallegada time not null,  
  horasalida time,  
  primary key(patente, horallegada) );
```

Nombramos los campos que formarán parte de la clave separados por comas.

Si vemos la estructura de la tabla vemos que en la columna "key", en ambos campos aparece "PRI", porque ambos son clave primaria.



Eliminar Agregar campos

Es posible eliminar un campo que es parte de una clave primaria, la clave queda con los campos restantes. Esto, siempre que no queden registros con clave repetida. Por ejemplo, podemos eliminar el campo "horallegada":

```
ALTER TABLE vehiculos DROP horallegada;
```

Para añadir campos a la tabla

```
ALTER TABLE vehiculos ADD horallegada time not null;
```



Campo con Autoincremento

Para establecer que un campo autoincremente sus valores automáticamente, éste debe ser entero (integer) y debe ser clave primaria:

```
CREATE TABLE libros( codigo int  
auto_increment, titulo varchar(20), autor  
varchar(30), editorial varchar(15), primary key  
(codigo) );
```

Cuando un campo tiene el atributo "auto_increment" no es necesario ingresar valor para él, porque se inserta automáticamente tomando el último valor como referencia, o 1 si es el primero. Para ingresar registros omitimos el campo definido como "auto_increment", por ejemplo:

```
INSERT INTO libros (titulo,autor,editorial)  
values('El aleph','Borges','Planeta');
```



Campo con Autoincremento

Está permitido ingresar el valor correspondiente al campo "auto_increment", por ejemplo:

```
INSERT INTO libros (codigo,titulo,autor,editorial)
values(6,'Martin Fierro','Jose Hernandez','Paidos');
```

Debemos tener en cuenta que:

- si el valor está repetido aparecerá un mensaje de error y el registro no se ingresará.
- si el valor dado saltea la secuencia, lo toma igualmente y en las siguientes inserciones, continuará la secuencia tomando el valor más alto.
- si el valor ingresado es 0, no lo toma y guarda el registro continuando la secuencia.
- si el valor ingresado es negativo (y el campo no está definido para aceptar sólo valores positivos), lo ingresa.



Campos Null

"null" significa "dato desconocido" o "valor inexistente". No es lo mismo que un valor 0, una cadena vacía o una cadena literal "null".

A veces, puede desconocerse o no existir el dato correspondiente a algún campo de un registro. En estos casos decimos que el campo puede contener valores nulos.

```
ALTER TABLE libros ADD precio null;
```

Si un campo acepta valores nulos, podemos ingresar "null" cuando no conocemos el valor.

```
INSERT INTO libros (titulo,autor,editorial,precio) values('El combustible espiritual','Ari Paluch ','Planeta', null);
```

```
INSERT INTO libros (titulo,autor,editorial,precio) values('El imperio digital ','Leandro Zanoni ','Ediciones B', 36);
```



Campos Not Null

Para que un campo no permita valores nulos debemos especificarlo luego de definir el campo, agregando "not null". Por defecto, los campos permiten valores nulos, pero podemos especificarlo igualmente agregando "null".

Para recuperar los registros que contengan el valor "null" en el campo "precio" no podemos utilizar los operadores relacionales vistos anteriormente: = (igual) y <> (distinto); debemos utilizar los operadores "is null" (es igual a null) y "is not null" (no es null):

```
SELECT from libros  
where precio is null
```

```
SELECT from libros  
where precio=0;
```



Order By

Podemos ordenar el resultado de un "select" para que los registros se muestren ordenados por algún campo

```
SELECT codigo,titulo,autor,editorial,precio from libros  
ORDER BY codigo;
```

También podemos colocar el número de orden del campo por el que queremos que se ordene en lugar de su nombre.

```
SELECT codigo,titulo,autor,editorial,precio from libros  
ORDER BY 4;
```



Funciones de Agrupamiento

La función "sum()" retorna la suma de los valores que contiene el campo especificado.

Por ejemplo, queremos saber la cantidad de libros que tenemos disponibles para la venta:

```
ALTER TABLE libros ADD cantidad integer;
```

```
SELECT sum(cantidad) from libros;
```

```
SELECT sum(cantidad) from libros;  
where editorial ='Planeta';
```

Para averiguar el valor máximo o mínimo de un campo usamos las funciones "max()" y "min()"

```
SELECT max(precio) from libros;
```



Count

Para averiguar la cantidad de registros sin necesidad de contarlos manualmente usamos la función "count()

```
SELECT count(*) from libros;
```

```
SELECT count(*) from libros  
where editorial='Planeta';
```

Para contar los registros que tienen precio (sin tener en cuenta los que tienen valor nulo), usamos la función "count()" y en los paréntesis colocamos el nombre del campo que necesitamos contar:

```
SELECT count(precio) from libros;
```



Uso de Varias Tablas

Hasta ahora hemos trabajado con una sola tabla, pero en general, se trabaja con varias tablas.

Para evitar la repetición de datos y ocupar menos espacio, se separa la información en varias tablas. Cada tabla tendrá parte de la información total que queremos registrar.

Por ejemplo, los datos de nuestra tabla "libros" podrían separarse en **2 tablas**, una "libros" y otra "editoriales" que guardará la información de las editoriales.

En nuestra tabla "libros" haremos referencia a la editorial colocando un código que la identifique.

```
SELECT (*) from libros;
```



Join

Cuando obtenemos información de más de una tabla decimos que hacemos un "join" (unión).

```
SELECT * from libros join editoriales on  
libros.codigoeditorial=editoriales.codigo;
```

Indicamos el **nombre de la tabla** luego del "from" ("libros"), **unimos** esa tabla con **"join"** y el nombre de **la otra tabla** ("editoriales"), luego especificamos la condición para enlazarlas con "on", es decir, el campo por el cual se combinarán. **"on"** hace coincidir registros de las dos tablas basándose en el valor de algún campo, en este ejemplo, los códigos de las editoriales de ambas tablas, el campo **"codigoeditorial"** de **"libros"** y el campo **"codigo"** de **"editoriales"** son los que enlazarán ambas tablas.



Join

Cuando se combina (join, unión) información de varias tablas, es necesario indicar qué registro de una tabla se combinará con qué registro de la otra tabla.

Si no especificamos por qué campo relacionamos ambas tablas, por ejemplo:

```
SELECT * from libros join editoriales;
```

el resultado es el producto cartesiano de ambas tablas (cada registro de la primera tabla se combina con cada registro de la segunda tabla), un "join" sin condición "on" genera un resultado en el que aparecen todas las combinaciones de los registros de ambas tablas. La información no sirve.



Join

Al nombrar el campo usamos el nombre de la tabla también.

Cuando las tablas referenciadas tienen campos con igual nombre, esto es necesario para evitar confusiones y ambigüedades al momento de referenciar un campo. En este ejemplo, si no especificamos "editoriales.codigo" y solamente tipeamos "codigo", MySQL no sabrá si nos referimos al campo "codigo" de "libros" o de "editoriales".

```
SELECT * from libros join editoriales on  
libros.codigoeditorial=editoriales.codigo;
```

